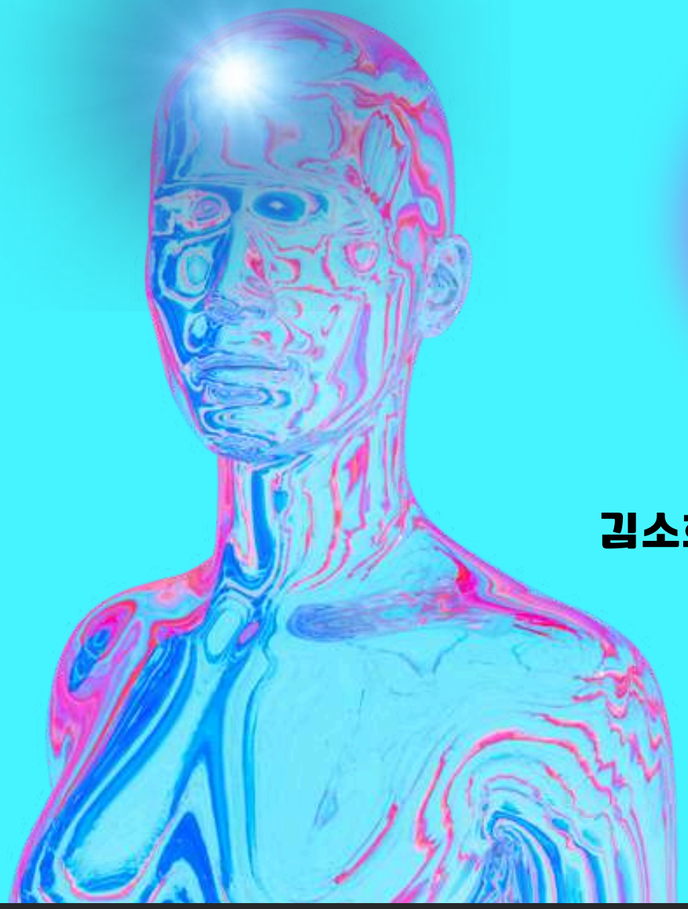
Video Image Audio Text

헌터 X 걸즈

Technology Future Intelligence

# FAKE HUNTERS



김소희 서희정 장하영 박소영

GET STARTED

Video Image Audio Text

FAKE HUNTERS

▲출처 : 미디어오늘(https://www.mediatoday.co.kr)

# 목차
# CONTENTS

프로젝트 개요

기술 스택 및 아키텍처

주요 기능

시연

기대효과 및 향후 고도화

Q&A

Video Image Audio Text

FAKE HUNTERS

# 프로젝트 개요

## PROJECT OVERVIEW

Video | Image | Audio | Text

# FAKE HUNTERS

[뉴스쏙] "딥페이크 피해 1위 국가는 한국…문제의 '
벌"…입법 추진 / 연합뉴스TV (YonhapnewsTV) 20
https://www.youtube.com/watch?v=UUx_lpFxlo

ㅐ 3배 급증..피해자 절반 '미
/outube.com/watch?v=UUx

IT 일반

## 딥페이크 성범죄 피해자 90% '1020 여성'…1년 새 피해 지원 2배

중앙디지털성범죄피해지원센터, 1년 간 딥페이크 성범죄 피해자 1807명 지원
전체 피해자 중에 여성이 97.1%…1020 여성 전체 90%, 10대 이하 가장 많아
"폐쇄형 소셜미디어가 범죄 도구로 활용되면서 피해가 대량화·장기화"

윤유경 기자 602@mediatoday.co.kr
입력 2025.08.31 15:37 수정 2025.08.31 22:39

N 기자구독 후원

https://www.mediatoday.co.kr/news/articleView.html?idxno=328483

# 프로젝트 목표

## 딥페이크 기술의 발전과 위협

- 생성형 AI 기술 발전으로 딥페이크 제작 난이도 하락
- 정치적 악용, 금융 사기, 개인 명예 훼손 등 사회적 문제 증가

## 기대 활용 방안

- SNS/미디어의 딥페이크 자동 탐지
- 보이스피싱 방지 시스템
- 언론사의 가짜 뉴스 검증 도구

## 탐지 기술의 필요성

- 육안/육이로 구분 불가능한 수준의 딥페이크 증가
- 실시간 탐지 및 자동화된 검증 시스템 필요

## 사회적 가치

- 딥페이크 범죄 예방 및 피해 최소화
- 디지털 콘텐츠 신뢰성 확보
- 정보 생태계 건전성 유지

# 팀원 소개

## 소희

- 딥페이크  영상 탐지
- 딥페이크 음성 AI/백엔드
- PPT 제작 및 발표

## 희정

- 메인페이지
- 딥페이크 이미지 탐지
- 딥페이크 음성 프론트
- PPT 제작

## 하영

- 가짜뉴스 텍스트 탐지

## 소영

- 프로젝트 아이디어

Video Image Audio Text

FAKE HUNTERS

# 프로젝트 목표

## 다양한 형태의 조작된 미디어 탐지

딥페이크
**영상** 탐지

딥페이크
**이미지** 탐지

**음성** 합성 및
변조 탐지

**가짜 뉴스** 판별

# 기술 스택

## FRONTEND

React JS

Tailwind CSS

## BACKEND

Java

spring boot

spring AI

MyBatis

## STORAGE

amazon S3

PostgreSQL

redis

## AI SERVICE

python

FastAPI

uvicorn

## TOOL & COLLABORATION

GitHub

Notion

Make

kaggle

ERD

Google Meet

Google Drive

Video Image Audio Text

FAKE HUNTERS

# 전체 아키텍처

**OPEN AI**
CHAT GPT
결과를 자연어로 설명

**FRONTEND**
REACT
파일 업로드, 결과 시각화

**BACKEND**
SPRING BOOT
비즈니스 로직

**STORAGE**
AWS S3 /RDS
미디어파일
분석결과 저장

**AI SERVICE**
FASTAPI
딥러닝 학습 모델로
딥페이크 분석

**CACHE**
REDIS (LOCAL)
실시간 분석 진행률

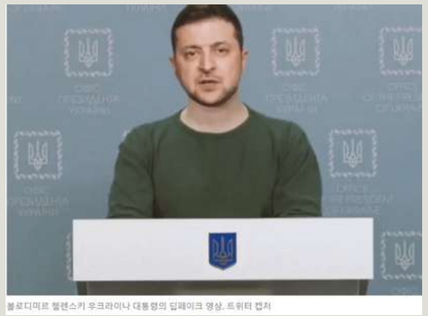
Video Image Audio Text

FAKE HUNTERS

# 주요 기능

## CORE FEATURES

# 영상 딥페이크

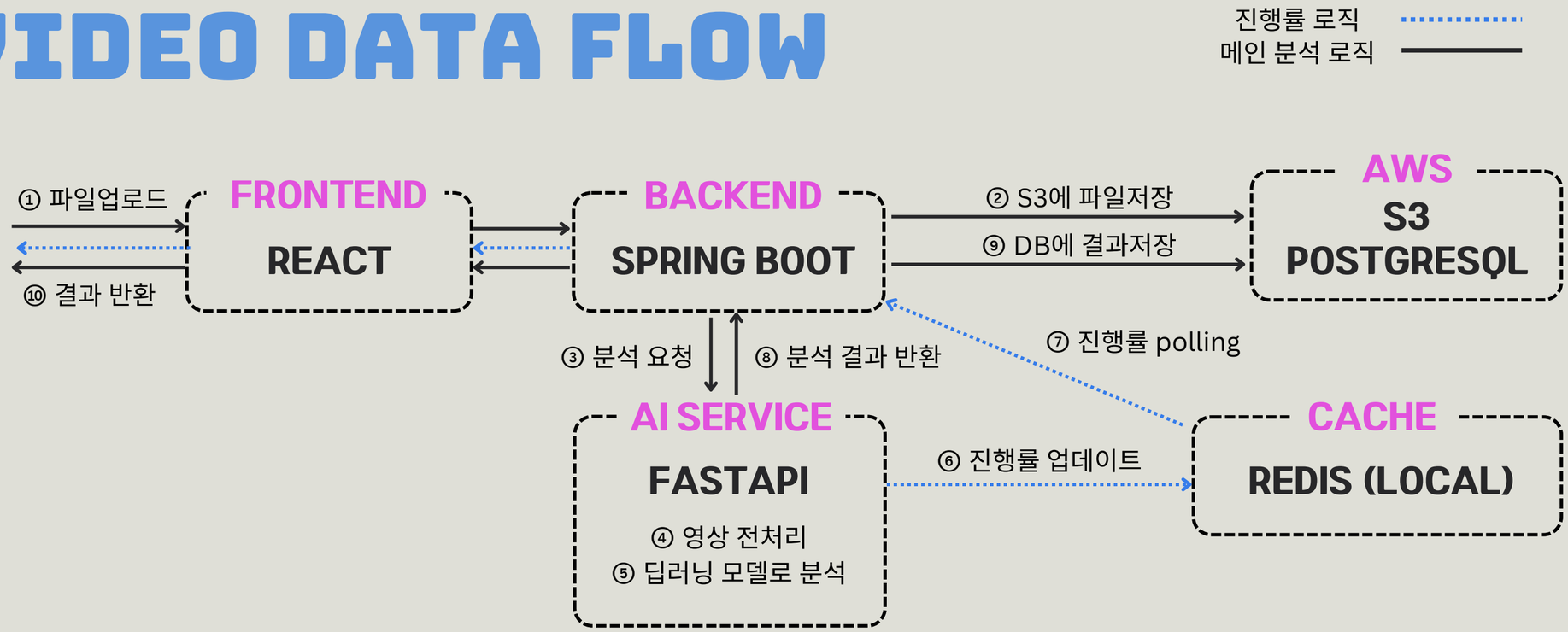


우크라이나 대통령이 러시아에 항복을 선언하는 듯한 가짜 동영상
아이유의 몸에 해외 배우 로버트 다우니 주니어의 얼굴을 합성한 영상
유명 연예인이 직접 홍보하는 것처럼 꾸민 광고 영상
자녀를 납치했다며 돈을 요구하는 협박 금품 탈취 범죄에 쓰인 영상

⋮

**영상 전체가 아니라 얼굴 영역에 주로 집중**

# VIDEO DATA FLOW



# 영상 분석 흐름

## 전처리 단계

OpenCV 라이브러리로
일정 간격마다 프레임 추출

MTCNN 라이브러리로
얼굴 검출과 정렬

ImageNet 정규화

동일한 크기로 리사이징

## 분석 단계

XceptionNet
– 공간적 아티팩트 탐지

EfficientNet-B4
– 구조적 이상 탐지

CNN-LSTM
– 시간적 일관성 분석

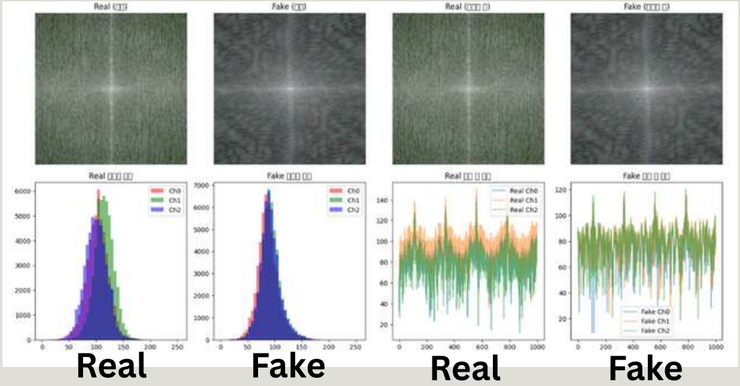
## 결과 단계

각 프레임의 분석 결과를
모델 별로 평균 계산

Soft Voting으로
평균값을 가중 평균
(0.4, 0.3, 0.3)

Fake/Real 판정

# 영상 전처리 상세




## MTCNN 딥러닝 기반의 알고리즘으로 데이터 추출을 담당

- 얼굴 검출 : 이미지 내에서 사람의 얼굴이 있는 위치를 찾아 바운딩 박스 좌표를 반환
- 신뢰도 계산 : 찾은 물체가 얼굴일 확률(probs)을 계산
- 랜드마크 추출 : 얼굴 내의 주요 특징점 5곳(왼쪽 눈, 오른쪽 눈, 코, 입술 왼쪽 끝, 입술 오른쪽 끝)의 좌표를 찾음

## OpenCV MTCNN이 찾아낸 데이터로 이미지 행렬을 조작하는 역할

- 색상 공간 변환 : OpenCV의 기본 BGR 형식을 딥러닝 모델용 RGB로 변환
- 기하학적 변환 : 양쪽 눈의 좌표를 이용해 얼굴이 기울어진 각도를 계산하고, 이를 회전시키기 위한 행렬을 생성하여 계산된 행렬을 적용해 이미지 정렬
- 이미지 프로세싱 : 회전된 이미지에서 얼굴 부분만 슬라이싱하여 추출하고, 신경망 입력 규격에 맞게 크기를 조절

# 딥페이크 탐지와 모델 선정

## XCEPTIONNET
미세한 공간적 아티팩트 탐지

얼굴 경계의 부자연스러운 블렌딩

피부 질감의 비현실적 매끄러움

눈/코/입 주변의 미세 왜곡

RGB 채널 간 불일치

**단독 성능: 91.75%**

## EFFICIENTNET-B4
다층적 구조 분석 탐지

다양한 해상도에서의 아티팩트

얼굴 전체의 구조적 불일치

조명 방향과 그림자 위치 불일치

색상/명암 분포의 비정상성

**단독 성능: 81.43%**

## CNN-LSTM
시간적 일관성 탐지

프레임 간 불연속적 변화

눈 깜빡임 패턴의 비정상성

움직임과 그림자의 불일치

시간적 아티팩트의 반복 패턴

**단독 성능: 90.00%**

## LFCC
다층적 구조 분석 탐지

주파수 대역별 에너지 불균형

GAN 모델 고유의 주파수 지문

블렌딩 경계의 불연속성

**단독 성능: 50.36%**

✕ 전처리 과정에서
공간 정보 손실로 실패

# 모델 학습 결과

```
lisso@DESKTOP-NIKJHOF MINGW64 ~/Desktop/ai-model/app/dom
ction (video/feat/ensemble-system)
$ # Best model 확인
python -c "import torch; ckpt = torch.load('weights/xcep
_location='cpu', weights_only=False); print('=== Best Mo
epoch')); print('Accuracy:', ckpt.get('metrics', {}).get
pt.get('metrics', {}).get('precision')); print('Recall:'
')); print('F1:', ckpt.get('metrics', {}).get('f1')); pr
et('auc'))"
=== Best Model ===
Epoch: 11
Accuracy: 0.9174603174603174
Precision: 0.9320557491289199
Recall: 0.8916666666666667
F1: 0.9114139693356048
AUC: 0.9730984848484848
```

```
Epoch 15/20
----------------------------------------------
Training:   0%|          | 0/603 [00:00<?, ?it/s]C:\Users\lisso\anaco
e-packages\torch\utils\data\dataloader.py:668: Us
e but no accelerator is found, then device pinned
  warnings.warn(warn_msg)
Training: 100%|##########| 603/603 [35:37<00:00,
Validation: 100%|##########| 158/158 [02:50<00:00

Train Loss: 0.0132, Train Acc: 0.9961
Val Loss: 0.3808
Val Accuracy: 0.9151
Val Precision: 0.9101
Val Recall: 0.9117
Val F1: 0.9109
Val AUC: 0.9682
Val Confusion Matrix:
[[606  54]
 [ 53 547]]
Learning Rate: 0.000147
```

```
Epoch [3/30]
Training: 100%|██████████| 350/350 [02:51<00:00,  2.04it/s, loss=0.6981]
  예측 분포: {np.int64(0): np.int64(13255), np.int64(1): np.int64(9145)}
Validation: 100%|██████████| 75/75 [00:23<00:00,  3.25it/s]
  예측 분포: {np.int32(1): np.int64(4800)}
Train - Loss: 0.6935, Acc: 0.4992, F1: 0.4487, AUC: 0.5025
Val   - Loss: 0.6933, Acc: 0.5000, F1: 0.6667, AUC: 0.5100
✓ 최고 성능 모델 저장! (Val F1: 0.6667)
```

```
Epoch [5/20]
------------------------------------------------------------
Training: 100%|##########| 500/500 [38:27<00:00,  4.61s/it, loss=0.75]
Validation: 100%|##########| 125/125 [02:13<00:00,  1.07s/it]
Train Loss: 0.0300, Acc: 0.9895
Val Loss: 0.2994
Val Accuracy: 0.9020
Val Precision: 0.9517
Val Recall: 0.8470
Val F1: 0.8963
Val AUC: 0.9747
Confusion Matrix:
[[957  43]
 [153 847]]
Elapsed time: 178.2 min
Saved: efficientnet_best.pth
New Best Model! Accuracy: 0.9020
Saved: efficientnet_epoch_5.pth
```

```
==================================================
Epoch 11/50
==================================================
Training: 100%|##########| 175/175 [24:09<00:00,  8.28
Validation: 100%|##########| 38/38 [01:53<00:00,  2.99

Epoch 11 Results:
  Train Loss: 0.2529, Train Acc: 89.36%
  Val Loss: 0.2666, Val Acc: 90.00%
  Learning Rate: 0.000100
  * New best validation accuracy!
Best model saved: weights\cnn-lstm\improved\best_model
Latest best link updated: weights\cnn-lstm\improved\be
```

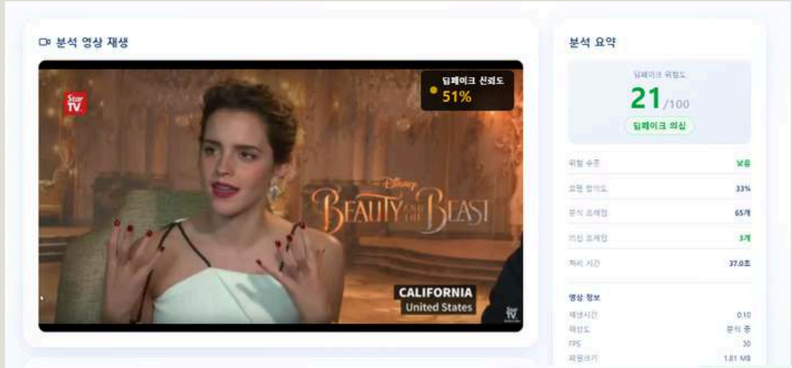
```
Epoch 12/50
------------------------------------------------------------
Training: 100%|██████████| 1525/1525 [22:20<00:00,  1.14it/s, l
Validation: 100%|██████████| 380/380 [01:25<00:00,  4.43it/s]

Train Loss: 0.0890, Acc: 0.9652
Val Loss: 0.1083
Val Accuracy: 0.9616
Val Precision: 0.9867
Val Recall: 0.9706
Val F1: 0.9786
Val AUC: 0.9808
Val Confusion Matrix:
[[ 1028   144]
 [  323 10661]]
LR: 0.000087
✓ Saved: efficientnet_best.pth (201.96 MB)

✓ Best model! Acc: 0.9616
```

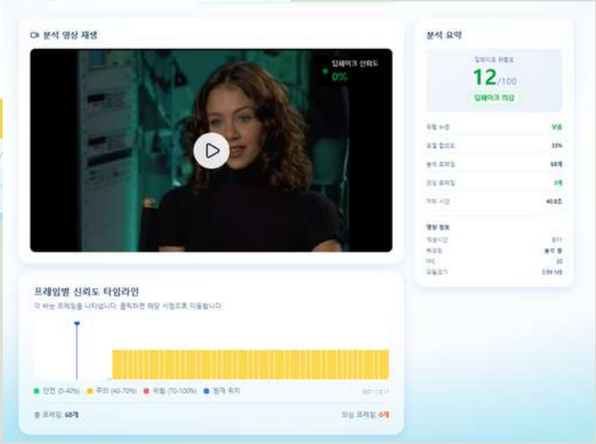
# 모델 학습 결과

| 모델 | Accuracy | Precision | Recall | F1 Score | AUC | 데이터 |
|---|---|---|---|---|---|---|
| XceptionNet | 91.75% | 93.21% | 89.17% | 91.14% | 97.31% | Celeb-DF 6,079개 |
| EfficientNet-B4 | 91.51% | 92.57% | 89.33% | 90.92% | 96.83% | Celeb-DF 6,079개 |
| CNN-LSTM | 90.00% | 85.50% | 82.80% | 84.13% | 94.50% | FaceForensics++ 2,000개 |

Video | Image | Audio | Text

# FAKE HUNTERS



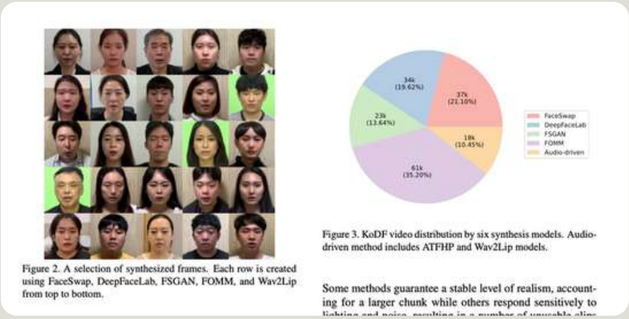


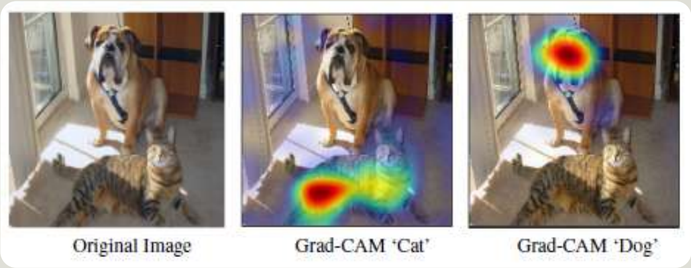
# IMAGE DEEPFAKE

## 목표

- 실제 환경에 가까운 데이터로 신뢰 가능한 이미지 딥페이크 탐지
- 단순 판별이 아니라 설명 가능한 결과(Grad-CAM) 제공

한국형 딥페이크 탐지 데이터 셋(KoDF)



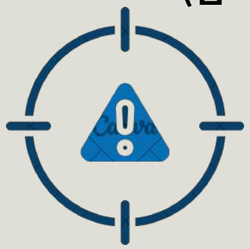

Grad-CAM

# IMAGE SERVICE FLOW

Input (입력)

Preprocessing (전처리)

Model Inference
(탐지)



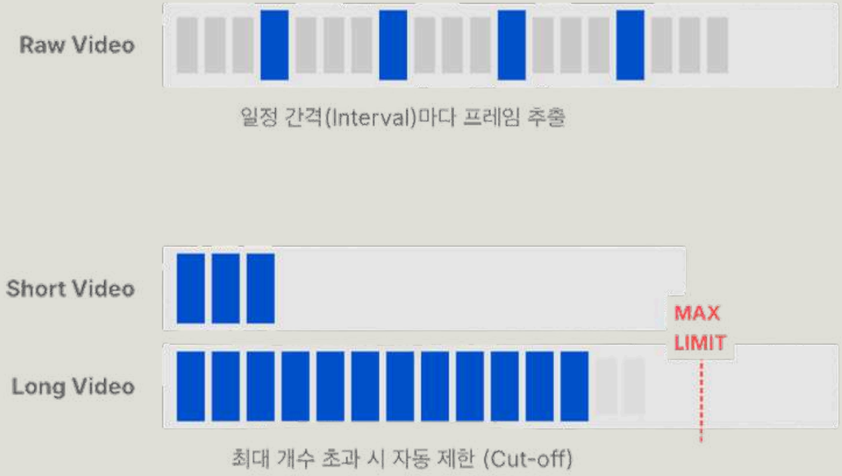
Explainability (판단근거시각화)

Result
Integration
(결과 통합 및 전달)

KoDF 데이터셋으로 학습한 Xception 모델을 활용해 이미지 딥페이크를 탐지하고 Grad-CAM으로 판단 근거를 시각화하는 End-to-End 분석 시스템

# HOW TRAINING PIPELINE

## DATA PREPROCESSING




중복과 편향을 최소화한 전처리 파이프라인을 설계

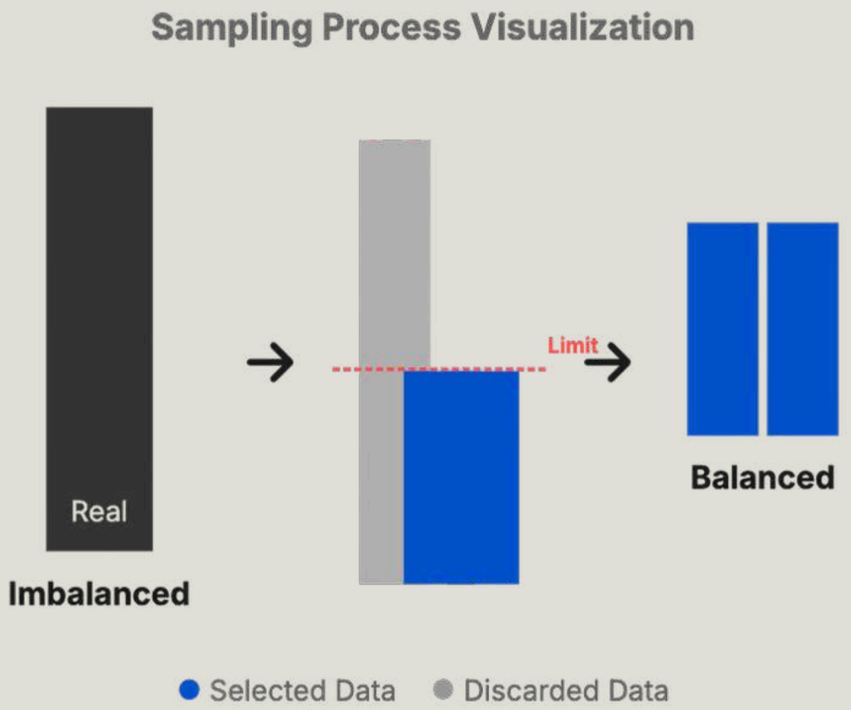
*Data Augmentation : 기존의 데이터를 이용해 새롭게 데이터를 창조하는 기법

# CLASS BALANCING

데이터 불균형 문제

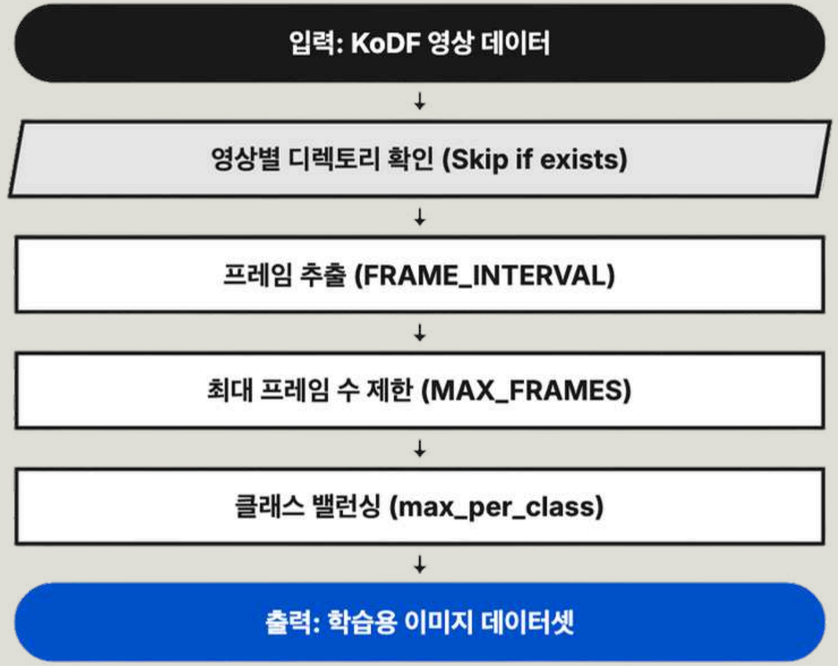
MAX_PER_CLASS 전략

기대 효과

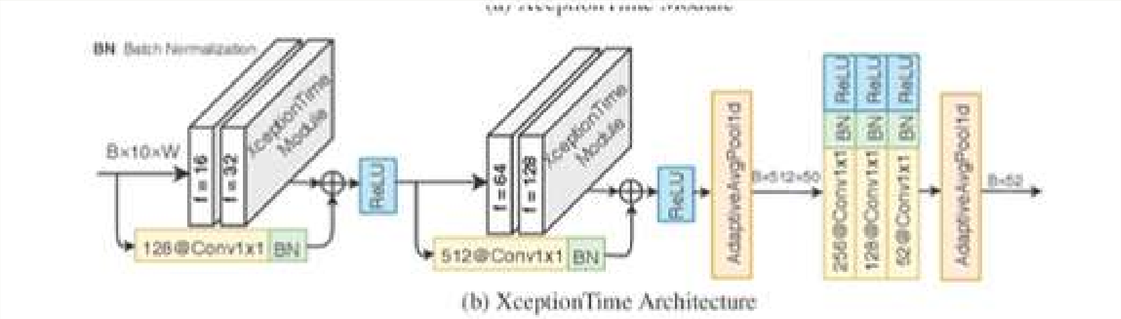


# HOW TRAINING PIPELINE

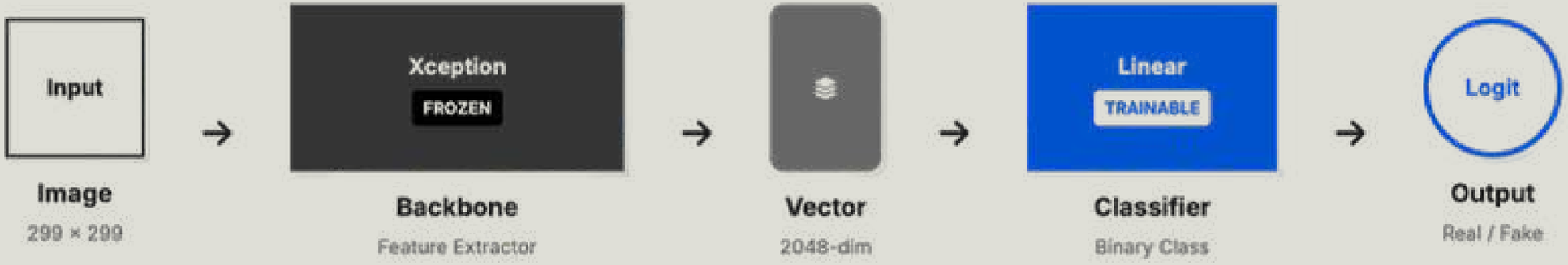
## DATA PREPROCESSING



# HOW TRAINING XCEPTION




(b) XceptionTime Architecture



Video | **Image** | Audio | Text

# FAKE HUNTERS

```
!python train.py

[RAW] real=161849, fake=37732
[USED] real=2000, fake=2000
/usr/local/lib/python3.12/dist-packages/
  model = create_fn(
🚀 Model initialized
✅ First batch loaded
[Epoch 1] loss=0.5702 val_auc=0.9487
🔥 Best model saved (AUC=0.9487)
[Epoch 2] loss=0.4296 val_auc=0.9733
🔥 Best model saved (AUC=0.9733)
✅ Training completed
```

**Epoch = 2**
**MAX_PER_CLASS = 2000**

```
!python train.py

[RAW] real=161849, fake=37732
[USED] real=5000, fake=5000
/usr/local/lib/python3.12/dist-packages/timm/
  model = create_fn(
🚀 Model initialized
✅ First batch loaded
[Epoch 1] loss=0.4862 val_auc=0.9635
🔥 Best model saved (AUC=0.9635)
[Epoch 2] loss=0.3364 val_auc=0.9860
🔥 Best model saved (AUC=0.9860)
✅ Training completed
```

**Epoch = 2**
**MAX_PER_CLASS = 5000**

```
+ 코드   + 텍스트

!python train.py

[RAW] real=161849, fake=37732
[USED] real=10000, fake=10000
/usr/local/lib/python3.12/dist-packages/timm/m
  model = create_fn(
Downloading: "https://github.com/rwightman/pyt
🚀 Model initialized
✅ First batch loaded
[Epoch 1] loss=0.3985 val_auc=0.9881
🔥 Best model saved (AUC=0.9881)
[Epoch 2] loss=0.2693 val_auc=0.9946
🔥 Best model saved (AUC=0.9946)
✅ Training completed
Figure(600x400)
```

**Epoch = 2**
**MAX_PER_CLASS = 10000**

# POLLING

대용량·시간 소요 분석을 서비스에서 어떻게 안정적으로 처리했는가

처리 시간 수 초 ~ 수십 초

**비동기 분석 구조 필요**

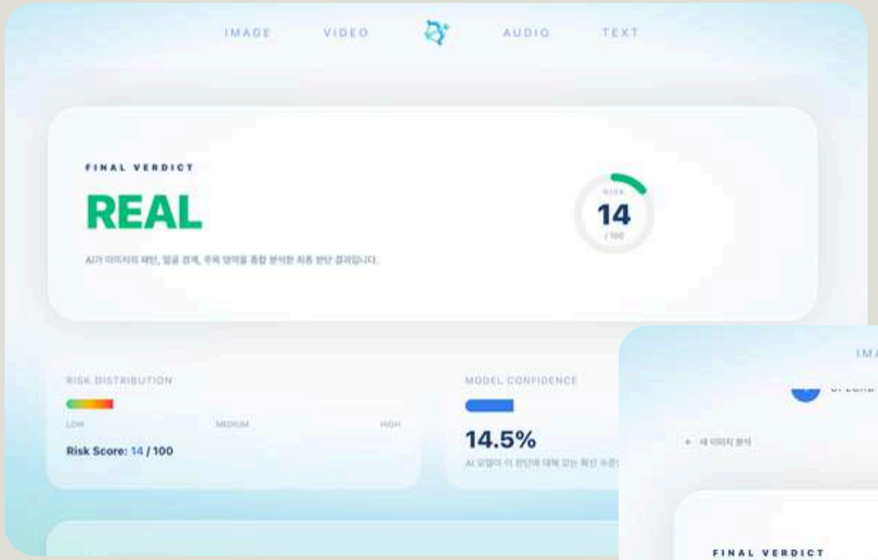
## 상태 기반 Polling 구조

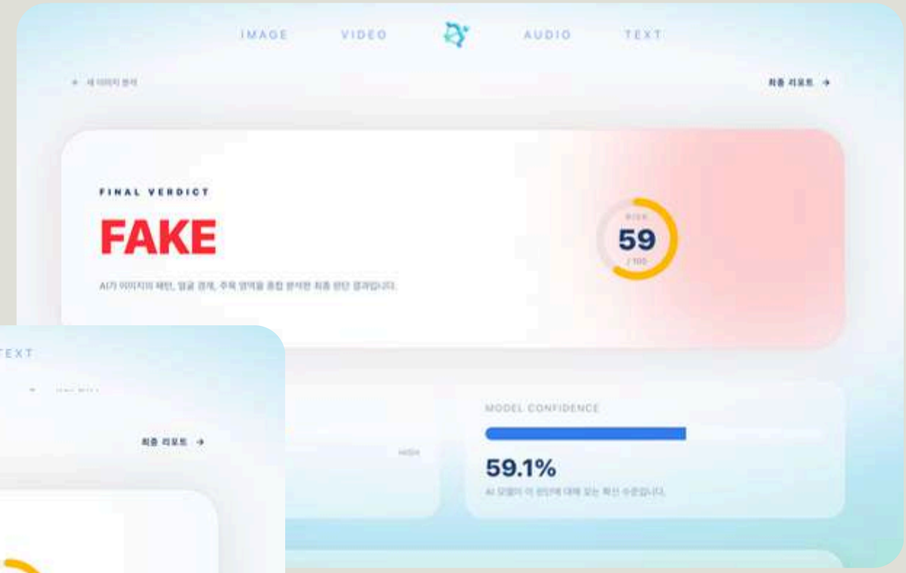
Audio: Hook 중심 Polling

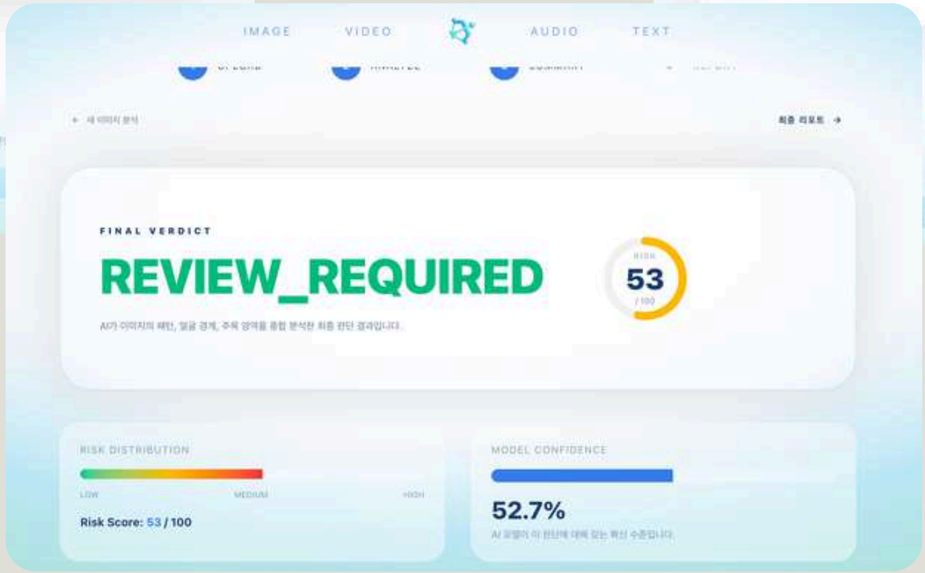
Image: Store(Zustand) 기반 Polling + 단계형 UX

Video | **Image** | Audio | Text

# FAKE HUNTERS



# 음성 딥페이크 탐지 모델

## LFCC CNN

선형 주파수 기반 고정밀 탐지 모델

✓ 고주파 아티팩트 (AI 합성 흔적)
✓ 위상 불일치 패턴
✓ 선형 주파수 대역의 비정상성
✓ Vocoder 특유의 아티팩트

**성능: Validation Accuracy 99.57%**

## MEL-SPECTROGRAM CNN

청각적 특징 기반 탐지 모델

✓ 저주파 대역의 음성 패턴 이상
✓ 성대 진동의 부자연스러움
✓ 음성 에너지 분포의 불균형
✓ TTS/VC 특유의 멜 패턴

**성능: Validation Accuracy 99.15%**

# 음성 데이터베이스 설계

오디오 파일 정보
audio_files
id bigserial
file_name varchar(255) NN
file_path varchar(512) NN
file_size bigint NN
duration decimal(10,2) NN
sample_rate integer NN
uploaded_at timestamp
status varchar(50)
created_at timestamp
updated_at timestamp
분석 결과
audio_analysis_results
id bigserial
audio_file_id bigint NN
prediction varchar(20) NN
confidence decimal(5,4) NN
real_probability decimal(5,4) NN
fake_probability decimal(5,4) NN
suspected_method varchar(100)
method_confidence decimal(5,4)
processing_time decimal(10,2)
analyzed_at timestamp
model_version varchar(50) NN
created_at timestamp
updated_at timestamp
시스템 설정
audio_system_settings
key varchar(100)
value varchar(255) NN
description text
updated_at timestamp
모델별 예측 결과
audio_model_predictions
id bigserial
analysis_result_id bigint NN
model_name varchar(100) NN
model_type varchar(50) NN
real_probability decimal(5,4) NN
fake_probability decimal(5,4) NN
prediction varchar(20) NN
confidence decimal(5,4) NN
created_at timestamp
스펙트로그램 히트맵 데이터
audio_spectrogram_heatmap
id bigserial
analysis_result_id bigint NN
heatmap_url varchar(512)
heatmap_type varchar(30) NN
created_at timestamp
시간대별 위험도
audio_time_segment_analysis
id bigserial
analysis_result_id bigint NN
start_time decimal(10,3) NN
end_time decimal(10,3) NN
risk_level varchar(20) NN
risk_score decimal(5,4) NN
reason text
indicators jsonb
created_at timestamp
주파수 분석 결과
audio_frequency_analysis
id bigserial
analysis_result_id bigint NN
frequency_band integer NN
anomaly_score decimal(5,4) NN
is_suspicious boolean
created_at timestamp
상세 탐지 근거
audio_detection_indicators
id bigserial
analysis_result_id bigint NN
indicator_type varchar(50) NN
description text NN
severity varchar(20) NN
confidence decimal(5,4) NN
time_start decimal(10,3)
time_end decimal(10,3)
created_at timestamp

# AUDIO MODEL TRAINING

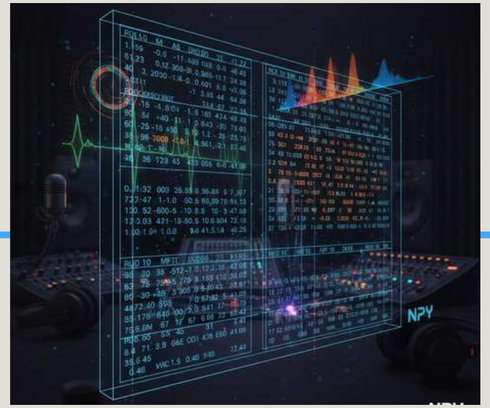
## 데이터셋 준비

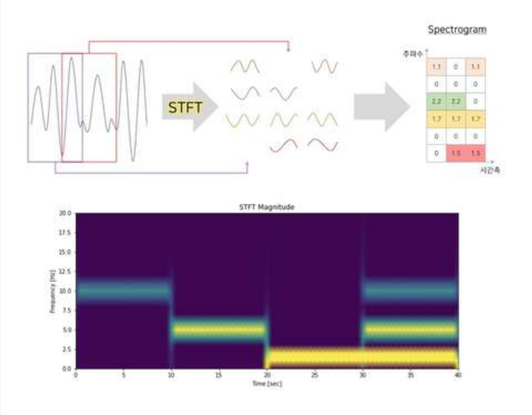
```
data
  processed\spectrograms
  raw\LA
    ASVspoof2019_LA_asv_protocols
    ASVspoof2019_LA_asv_scores
    ASVspoof2019_LA_cm_protocols
    ASVspoof2019_LA_dev
    ASVspoof2019_LA_eval
    ASVspoof2019_LA_train
    README.LA.txt
  splits
```

**ASVspoof2019**

## 전처리

- .flac 오디오 파일을 16kHz 샘플링 레이트로 로드
- 4초 길이로 통일 (길면 자르고, 짧으면 패딩)
- Mel-Spectrogram 계산 (64 mel bins 사용)
- 크기를 64x256로 조정하여 일관된 입력 크기 확보
- .npy 파일로 저장하여 학습 시 빠른 로딩



# TRAINING

**과적합 첫 신호 포착**

Epoch 4: Train 99.09%, Val 97.57%

**Learning Rate Scheduler가 작동**

Learning Rate가 0.0005로 감소

Train 99.65%, Val 99.15% (New Best)

Epoch 1 2 3 4 5 6 7 8 9

**안정적인 학습이 진행**

- Epoch 1: Train 93.62%, Val 97.10%
- Epoch 2: Train 97.96%, Val 96.18%
- Epoch 3: Train 98.88%, Val 98.54% (Best)

**과적합 심화**

- Val Loss: 0.0433 → 0.0637 → 0.0934 로 급증
  Val 정확도가 98.54% → 96.21% → 93.22%로 하락

# Goal

**뉴스 텍스트를 입력받아 조작·왜곡 가능성이 있는지 자동으로 분석하고, 판단 근거와 추가 확인 자료를 함께 제공하는 가짜뉴스 탐지 서비스 구현**

- AI 분류 결과 (FAKE/REAL)
- 분류에 대한 신뢰도(confidence) :

  confidence가 높을수록 FAKE, REAL 확률이 높음
- 의심되는 문장 하이라이트
- 판단 근거 문장(evidence)
- 외부 참고 자료(reference)

# Service Flow

사용자가 뉴스 텍스트 입력

→

**프론트 엔드 → 백엔드 API 호출**
POST /api/v1/text/analyze

→

**백엔드가 AI 추론 서버(FastAPI)로 전달**

**AI 서버**

텍스트 분류
근거 문장 추출
하이라이트 생성
참고자료 검색

→

**결과를 벡앤드로 반환**

↓

**DB에 결과 저장**
(INSERT-only)

→

**프론트 엔드**
**(사용자가 보는 화면)**

판정 결과(label, score)
입력 문장에 하이라이트 표시
참고자료 링크 출력

# Training Pipline

데이터 → 모델 → 평가 → 저장

**데이터 수집**
AIHub Clickbait 데이터셋(JSON)

→

**전처리**
title + content 결합
label(clickbaitClass) 추출
중복 제거 및 샘플링

→

**HuggingFace Dataset 변환**

→

**Tokeniing (BERT tokenizer)**

**모델 학습**

→

**성능 평가 (F1, accuracy)**

→

**모델 export 및 저장**

# Model

## 사용모델

**Base Model**
klue/bert-base

**Task**
이진 분류
(2-class classification)

**구조**
AutoModelForSequenceClassification

## 출력

**label**
0 or 1

**score**
softmax 확률 기반 점수

## 달라진 점

학습 데이터의 라벨 변경
Fake/Real
(허위여부)
↓
Clickbait / Non-clickbait
(선정성/과장성/낚시성 탐지모델)

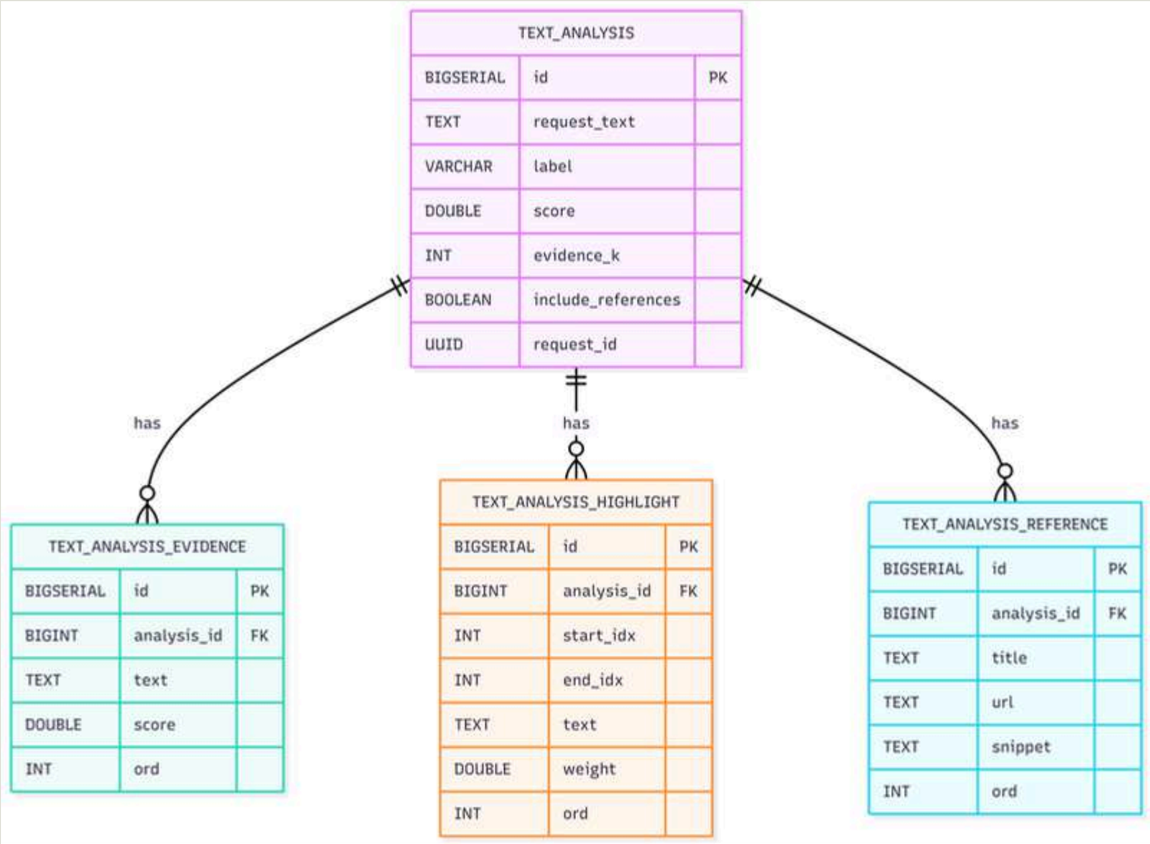
# Dataset

- AIHub, Naver뉴스 API를 통해 얻은 뉴스 데이터를 JSON 형식에서 CSV 형식으로 변환
- 뉴스 제목과 본문을 하나의 입력 텍스트로 결합
- Reservoir Sapmling 이용하여 약 50,000건의 학습 데이터를 구성
- 학습 데이터는 train/validation 세트로 분할 (9:1 비율)

# 전처리 과정

- 뉴스 텍스트에서 HTML 태그와 특수문자 제거
- 불필요한 공백과 줄바꿈을 정규화
- 토그나이저를 이용해 input_ids, attention_mask로 변환
- 최대 길이를 초과하는 문장은 자동으로 잘라서 처리

# Database 설계


TEXT_ANALYSIS
BIGSERIAL id PK
TEXT request_text
VARCHAR label
DOUBLE score
INT evidence_k
BOOLEAN include_references
UUID request_id
has
has
has
TEXT_ANALYSIS_EVIDENCE
BIGSERIAL id PK
BIGINT analysis_id FK
TEXT text
DOUBLE score
INT ord
TEXT_ANALYSIS_HIGHLIGHT
BIGSERIAL id PK
BIGINT analysis_id FK
INT start_idx
INT end_idx
TEXT text
DOUBLE weight
INT ord
TEXT_ANALYSIS_REFERENCE
BIGSERIAL id PK
BIGINT analysis_id FK
TEXT title
TEXT url
TEXT snippet
INT ord

Video Image Audio Text

FAKE HUNTERS

# 시연

## LIVE DEMO

Video Image Audio Text

FAKE HUNTERS

IMAGE VIDEO AUDIO TEXT

# FAKE HUNTERS

# 기대효과 및 향후 고도화

## EXPECTED EFFECTS AND FUTURE ENHANCEMENTS

# 기대효과

언론·미디어, SNS·플랫폼, 공공기관, 기업 등 다양한 분야에서 활용가능한 서비스

**멀티모달 통합 분석**

단일 웹 플랫폼에서 분석 가능

+

**실서비스 확장 가능성**

연구용 모델이 아닌,
실제 서비스 가능한 구조

+

**신뢰성 향상**

사용자와 운영자가
판단을 이해하고 신뢰 가능

+

**사회적·산업적 활용 가능성**

- 허위 정보 확산 방지
- 디지털 콘텐츠 신뢰도 회복
- AI 오남용에 대한
- 기술적 대응 사례 제시

# 향후 고도화
# VEDIO

## Transformer 기반 모델 도입

→ Self-attention으로 전역적 패턴 학습
→ 영상 특화 Transformer, SOTA 성능
→ 시공간 분리 attention, 효율적 학습

**기대 효과 : 93.1% → 96%+**

## Multi-Modal 분석 통합

입 모양은 자연스럽지만 음성이 불일치
→ Fake

음성은 진짜지만 표정 변화가 부자연
→ Partial Fake

## 적대적 공격 방어 전략 도입

→ 공격 샘플로 학습하여 강건성 확보
→ 다양한 모델 조합으로 우회 방지
→ JPEG 압축, 노이즈 제거 등 전처리

**의도적 회피 기법도 탐지 가능**

# 향후 고도화 IMAGE

## 멀티 모델 앙상블

- ex) 이미지에서 객체가 지워졌거나 인페인팅과 같은 조작 흔적이 있는지 탐지하는 기능

## 증명 인증서 발급

- 촬영 즉시 GPS + 타임스탬프 + 디바이스 정보 블록체인 저장 -  인증서 발급
  ex) 보험: "사고 현장을 조작 안 했음을 증명"

## 리포트 저장 & 공유 링크

## 안티 딥페이크 쉴드

## 보험 서비스

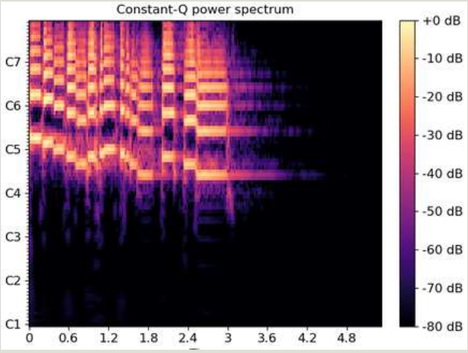
- 공개 웹만 타겟하고, 나머지는 "신고 도움" 기능으로 우회해서 딥페이크 크롤링

## 커뮤니티 검증 시스템

# 향후 고도화
# AUDIO

## Constant-Q Transform (CQT)

음악적 주파수 분석 강화 모델




## 한국어 지원

**언어 독립적 특징 추출**

→ Prosody, Pitch, Energy 분석

**Transfer Learning**

→ 영어 모델 → 한국어 Fine-tuning

# 향후 고도화
# TEXT

## 다국어 뉴스 분석 모델로 확장

**한계**

**번역 → 외신 검색 기반 교차 검증 파이프라인을 시도했으나 성공이 일관적이지 않았음**

쿼리 민감도/ 호출제한/ 검색 0건 문제로 재현성이 낮아 학습데이터 자동생성 실패

**개선**

1) 문장 번역이 아닌, NER 기반 엔티티, 이벤트 키워드 추출(다국어 공통 토큰)으로 전환
2) 검색소스를 Gnews 단일에서 확장(NewsAPI/Google CSE/SerpAPI 등),결과 0건 방지 fallback 전략 강화
3) 외신 여부로 라벨을 직접 만들지 않고, 참고자료 제공용으로만 사용하거나, 팩트체크 라벨 데이터로 정답을 확보한 뒤 학습

## 가짜 뉴스 모델 정교화

실제 가짜 뉴스 라벨 기반 모델 재학습

Video Image Audio Text

FAKE HUNTERS

# Q&A

Video Image Audio Text

헌터 X 걸즈

END PRESENTATION

김소희 박소영 서희정 장하영

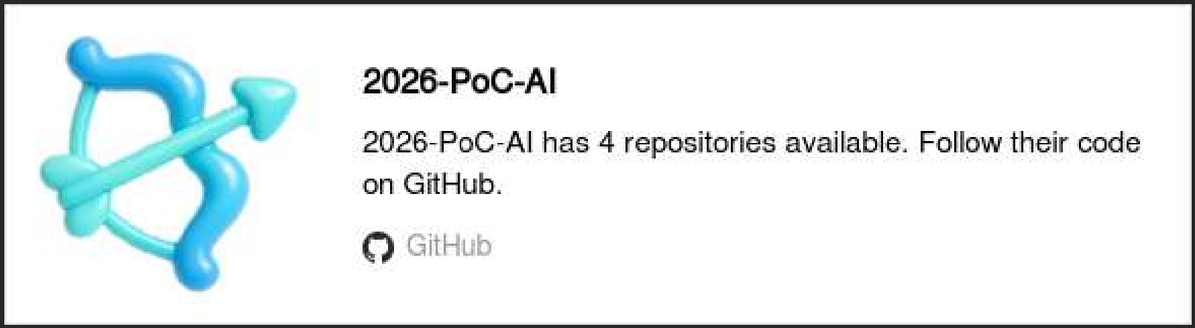
Technology Future Intelligence

# THANK YOU

## FOR YOUR ATTENTION

www.FAKEHUNTERS.com FAKE@HUNTERS.com +FAKE-HUNTERS

# FAKE HUNTERS 작업 GITHUB




https://github.com/2026-PoC-AI